```
$ bundle exec rspec spec/unit/app/api_spec.rb

Randomized with seed 34086

ExpenseTracker::API
  POST /expenses
    when the expense fails validation
      returns an error message (PENDING: Not yet implemented)
      responds with a 422 (Unprocessable entity) (PENDING: Not yet ↩
      implemented)
    when the expense is successfully recorded
      responds with a 200 (OK) (PENDING: Not yet implemented)
      returns the expense id (PENDING: Not yet implemented)

« truncated »

Finished in 0.00107 seconds (files took 0.15832 seconds to load)
4 examples, 0 failures, 4 pending

Randomized with seed 34086
```

It's time to fill in the behavior.

Filling In the First Spec

You don't have any classes or methods yet for indicating whether recording an expense succeeded or failed. Let's take a moment to sketch out what that code would look like.

Connecting to Storage

First, you'll need some kind of storage engine that keeps the expense history; call it a `Ledger`. The simplest approach would be for the `API` class to create a `Ledger` instance directly:

05-unit-specs/03/expense_tracker/api_snippets.rb

```
class API < Sinatra::Base
  def initialize
    @ledger = Ledger.new
    super() # rest of initialization from Sinatra
  end
end

# Later, callers do this:
app = API.new
```

But this style limits the code's flexibility and testability, as it doesn't allow you to use a substitute ledger for custom behavior. Instead, consider structuring the code so that callers pass an object filling the `Ledger` role into the `API` initializer: